Họa.

Hướng dẫn bảo vệ tiểu luận

Phân tích và thiết kế một cửa hàng tranh có AI: từ yêu cầu đến code chạy được

Môn Phân tích và thiết kế hệ thống thông tin · Lớp 04 · Vũ Hùng Anh (B23DCDT022) · 18:00 thứ Năm 12/11/2026

1. Tổng quan ngắn

Trình bày gì. Một mình em kể lại một sản phẩm, cửa hàng tranh Họa., đi qua đủ năm bước của môn: yêu cầu, phân tích, thiết kế, code, kiểm chứng. **Không dùng slide.** Công cụ chính là trang **Technical Defense Showcase**; khi thầy muốn xem tận nơi thì mở thẳng hiện vật thật bên dưới. Lời nói bằng tiếng Việt, mọi hiện vật bằng tiếng Anh.

Hiện vật	Mở ở đâu	Dùng để
Showcase (mục A–G)	`showcase/index.html`, qua `http://localhost:8000/showcase/`	đường đi chính của bài nói; mỗi mục có liên kết tới hiện vật gốc
Project Visual Paradigm	`vp/tieuluan_design.vpp` (40 sơ đồ)	cho thầy thấy sơ đồ là bản gốc, mở và sửa được
IDE (VS Code)	`tools/model.py` và vài file Java (mục 2.2)	chỉ đúng dòng code của một phương thức trong thiết kế
Cơ sở dữ liệu	`demo/data/ecom.db` (sqlite-web hoặc `sqlite3`)	bảng `category`, `product` và khóa ngoại thật
Cửa hàng chạy thật	`http://127.0.0.1:8686` (bản tắt AI: cổng 8687)	demo trực tiếp (mục 6)
Bài tiểu luận	`build/report/tieuluan_V3_04_10_VUHUNGANH_022.pdf`	tra khi bị hỏi sâu (mục 8.2)

Ba điều thầy nên nhớ sau buổi bảo vệ.

1. **Một đường liền từ vấn đề đến code.** Mỗi yêu cầu có use case, mỗi use case có sơ đồ, mỗi sơ đồ thành lớp và bảng, mỗi lớp và bảng có dòng code và bài kiểm thử. Mọi tên gọi chỉ viết một lần trong một bản đặc tả, và một công cụ tự so code với bản đặc tả: 185 phương thức, 91 cột đều khớp; 67 bài kiểm thử và 17 bước chạy thử đầu cuối đều đạt.
2. **Cơ sở dữ liệu làm theo góp ý của thầy và được sinh ra từ mô hình.** Danh mục, một bảng sản phẩm chung, và bảng chi tiết cho từng loại. Visual Paradigm sinh câu lệnh tạo bảng; em so với cơ sở dữ liệu đang chạy: giống hệt, chỉ thiếu khóa ngoại và ràng buộc kiểm tra mà bộ sinh của VP cho SQLite không viết, và em bổ sung chúng từ cùng mô hình.
3. **AI giúp khách nhưng không nắm quyền; tắt AI cửa hàng vẫn bán.** Giá, số hàng, quyền xem đơn do luật của cửa hàng quyết. Em nói thật kết quả: Gemma hơn rõ khi tìm bằng ảnh (75% so với 12%), nhưng tìm bằng chữ thì ngang luật đơn giản (93% cả hai).

Thời gian. Hỏi thầy thời lượng ngay khi lên. Bảy bước của mục 3, hai phương án:

	Bước	Mở chính	6–7 phút	3–4 phút
1	Vấn đề và yêu cầu	showcase A	45 giây	20 giây
2	Các sơ đồ chính	showcase B, VP: use case, activity	1 phút	40 giây
3	Từ phân tích sang thiết kế	showcase C, VP: lớp, ba tầng	45 giây	(gộp vào 2)
4	Cơ sở dữ liệu	showcase E, VP: ERD, bảng `category`	50 giây	30 giây
5	Từ đặc tả đến code	showcase D, IDE	45 giây	20 giây
6	Demo trực tiếp	cửa hàng	1 phút 45	1 phút
7	Kiểm thử và kết luận	showcase G	30 giây	20 giây
	Tổng		**≈ 6 phút 20**	**≈ 3 phút 10**

Lời nói của bảy bước dài khoảng 702 từ, tức chừng 4,4 phút ở tốc độ 160 từ mỗi phút; phần còn lại là thời gian chuyển cửa sổ và thao tác. Nếu thầy ngắt để hỏi: trả lời ngắn, rồi đi tiếp bước sau, không cần nói lại từ đầu.

2. Chuẩn bị

Mọi lệnh chạy từ **thư mục dự án** `~/PTIT/infor_sys/tieuluan` (bản trên GitHub là thư mục `Hoa`).

2.1 Tối nay hoặc sáng mai (làm một lần)

- ☐ **Biên dịch và kiểm thử:** `cd demo && ./build.sh && ./test.sh`. Phải thấy 67 dòng PASS và dòng cuối "…all match the model".
- ☐ **Gemma:** cấu hình hiện tại gọi Gemma 4 (bản 31B) qua dịch vụ **DeepInfra** (`demo/config/app.properties`); key nằm trong `demo/config/app.local.properties`, không có trên GitHub. Tối 11/11 máy chủ Gemma của lớp trả lỗi 502, chỉ người quản lý máy chủ mới bật lại được, nên cấu hình đã được đổi sang DeepInfra. Cách kiểm tra ở mục 2.3.
- ☐ **Whisper và micro:** lần đầu `./run.sh` tải mô hình (cần Internet một lần). Mở Chrome ở `http://127.0.0.1:8686`, bấm micro, chọn *Allow*, nói thử "blue landscape paintings". Chỉ nói **tiếng Anh**.
- ☐ **Ảnh demo** chép ra Desktop: `demo/data/eval_images/Q02.jpg` (The Great Wave) và `demo/web/img/rooms/room1.jpg`.
- ☐ **sqlite-web** (trình xem cơ sở dữ liệu đã dùng để chụp ảnh trong bài) **hiện chưa cài trên máy**. Cài vào một môi trường riêng: `python3 -m venv ~/sqlite-web && ~/sqlite-web/bin/pip install sqlite-web` (cần Internet). Không cài được thì dùng `sqlite3` có sẵn (mục 2.2).
- ☐ **Tập kịch bản 7 bước ba lần** có bấm giờ, rồi tập demo (mục 6), rồi **đặt lại dữ liệu mẫu** (mục 2.4).
- ☐ **Dự phòng:** ảnh chụp màn hình `figures/demo/*.png`; showcase cũng có trên GitHub Pages (`https://scalliontor.github.io/Hoa/showcase/`, cần Internet, có thể cũ hơn bản trên máy).
- ☐ **Máy:** sạc đầy, mang sạc và cáp máy chiếu, bật Không làm phiền, chạy `caffeinate -d` để máy không ngủ.

2.2 Trước giờ bảo vệ 30 phút: mở sẵn các cửa sổ

1. **Terminal 1, cửa hàng:** `cd demo && ./run.sh`. Lệnh này tự bật Whisper (nhật ký ở `demo/data/voice-server.log`) rồi bật cửa hàng. Chờ hai dòng `Họa. is running at http://127.0.0.1:8686` và `Gemma 4: online | Whisper: online`. Whisper cần khoảng 10 giây để nạp; kiểm tra bằng `curl http://127.0.0.1:9000/health`.
2. **Terminal 2, bản tắt Gemma:** `cd demo && ./run.sh config/app.offline.properties`. Chạy ở cổng 8687, cơ sở dữ liệu riêng, địa chỉ Gemma cố ý trỏ vào chỗ không có gì.
3. **Terminal 3, showcase:** ở thư mục dự án chạy `python3 -m http.server 8000 --bind 127.0.0.1`.
4. **Chrome, ba tab, phóng to 110–125%:** (1) showcase `http://localhost:8000/showcase/`; (2) cửa hàng `http://127.0.0.1:8686`, đăng nhập C001; (3) bản tắt Gemma `http://127.0.0.1:8687/?debug=1`.
5. **Visual Paradigm:** mở `vp/tieuluan_design.vpp` bằng **Visual Paradigm CE** và mở sẵn các sơ đồ dưới đây thành tab (tìm theo tên trong Diagram Navigator; tên bắt đầu bằng số mục của bài). Khi đóng VP, chọn *Don't Save*.

Tên sơ đồ trong VP	Dùng ở
"1.2 Use Case Diagram - Hoa Gallery"	bước 2
"2.2 Activity - Checkout"	bước 2
"2.8 Domain Model - analysis class diagram"	bước 3
"4.3 Class Diagram in Design"	bước 3; QĐ2, QĐ4
"4.4a Three Layers - classes and their links"	bước 3; QĐ3
"4.2 Database - entity relationship diagram"	bước 4; câu hỏi về khóa
"4.2 Physical Data Model - tables, keys and foreign keys"	câu hỏi về mức vật lý
"2.6b Analysis Sequence - Checkout (UC07)" và "4.4d Checkout across the layers (design sequence)"	câu hỏi phân tích khác thiết kế
"2.4 State Machine - Order"	QĐ4
"4.4a Three Logical Components", "4.4c Package Diagram - Design Packages"	câu hỏi kiến trúc

Bản Community mở được **mọi sơ đồ UML và ERD**. Năm sơ đồ BPMN, ArchiMate và DFD ("1.3 Stakeholder Map (ArchiMate)", "1.4 Business Process - Order to Delivery (BPMN)", "1.6 Problem Tree (ArchiMate)", "1.7 Context Diagram - DFD Level 0", "2.9c Data Flow Diagram - Level 1") là ảnh chụp bản dùng thử. Bản dùng thử **không mở lại được** file `vp/tieuluan_design.vpp` (báo "Evaluation on this project has been expired"), nên đừng mở

project bằng nó. Khi thầy hỏi đến năm sơ đồ này, mở ảnh chụp trong bài hoặc trong showcase (mục cuối "All diagrams").

6. **VS Code**: `open -a "Visual Studio Code" .` ở thư mục dự án. Mở sẵn các tab sau (Cmd+P gõ tên file, Ctrl+G gõ số dòng):

File	Dòng cần chỉ
`tools/model.py`	`REQUIREMENTS` 18, `ANALYSIS` 141, `DESIGN` 235 (lớp `Order` 359), `LAYERS` 742, `TABLES` 773
`Order.java`	`cancel` 41, `advanceTo` 47, `isOwnedBy` 55
`CheckoutService.java`	`checkout` 45; đọc lại giá `currentProduct` 100
`OrderService.java`	`getOrder` 21, `cancelOrder` 40, `updateStatus` 51
`SqliteProductRepository.java`	`decreaseStock` 70, `toProduct` 97 (chọn bảng chi tiết)
`ecom_schema_sqlite.sql`	`category` 48, `product` 57, `cart` 115, `cart_item` 125, `order_item` 147
`schema_comparison.txt`, `test_results.txt`	kết quả so sánh với VP; dòng cuối của lần chạy kiểm thử

   Cmd+P tìm theo tên file là đủ. Java nằm trong `demo/src/main/java/com/ptit/ecom/`, lược đồ trong `demo/db/`, hai file kết quả trong `build/`.

7. **Cơ sở dữ liệu** (chỉ đọc, mở song song với cửa hàng vẫn được): `~/sqlite-web/bin/sqlite_web --read-only demo/data/ecom.db` rồi mở bảng `category` và `product`. Không có sqlite-web thì:

```
sqlite3 -readonly -header -column demo/data/ecom.db
SELECT * FROM category;
SELECT product_id, category_id, name, price, stock_quantity FROM product LIMIT 6;
PRAGMA foreign_key_list(product);
```

2.3 Gemma có trả lời không? Quyết định trước khi vào phòng

1. **Phép thử quyết định:** ở tab cửa hàng nhấn **Shift+D**, tìm `blue landscape paintings`: nhãn phải là **GEMMA 4**. Nếu là **KEYWORD FALLBACK** thì Gemma không trả lời (lần tìm này đã dùng luật từ khóa). Dòng `Gemma 4: online` lúc khởi động chỉ là kiểm tra nhanh; hỏi thêm trợ lý `find my order 20261001` cho chắc.
2. Không trả lời: kiểm tra Internet (mạng trường, 4G), key trong `app.local.properties`, rồi khởi động lại cửa hàng (Ctrl+C, `./run.sh`).
3. Nếu chủ máy chủ của lớp đã bật lại Gemma và muốn dùng lại máy đó: sửa hai dòng trong `demo/config/app.properties` về `gemma.baseUrl=http://171.226.10.121:8000/llm/v1/` và `gemma.model=gemma-4`, đổi key trong `app.local.properties` sang key cũ (dòng đang bị chú thích), rồi khởi động lại. **Không** đặt `gemma.baseUrl` trong `app.local.properties`: bản tắt Gemma cũng đọc file đó và sẽ không còn "tắt".

DeepInfra vẫn lỗi: trình bày chế độ dự phòng một cách có chủ đích

Nếu DeepInfra không trả lời (mất mạng, dịch vụ lỗi) và máy chủ của lớp cũng chưa bật lại thì không giấu. Mở đầu demo bằng câu: *"Hôm nay dịch vụ Gemma không trả lời. Đây đúng là tình huống em đã thiết kế cho: cửa hàng vẫn tìm được tranh, vẫn đặt được hàng, chỉ là phần hiểu câu và hiểu ảnh chuyển sang luật đơn giản."* Đó là hai yêu cầu chất lượng NFR02 (dự phòng) và NFR12 (cửa hàng chạy được khi không có AI). Phần nào cần Gemma thì chiếu ảnh chụp (mục 6.6).

2.4 Tài khoản demo và đặt lại dữ liệu

Tài khoản	Loại	Dữ liệu mẫu
C001 Nguyen Minh An	khách VIP, giảm 10%	đơn 20261001 (đã gửi), 20261002 (đã giao); giỏ có sẵn Woman with a Parasol
C002 Tran Thu Binh	khách mới	đơn 20261003 (mới đặt)
ADMIN Gallery staff	quản trị viên	menu *Back office*: tranh, khung, trạng thái đơn

Đổi tài khoản bằng cách bấm tên ở góc phải trên. Mỗi lần tập làm đổi dữ liệu (đơn mới, kho giảm). Trước giờ bảo vệ: dừng hai cửa hàng (Ctrl+C), rồi

```
rm -f demo/data/ecom.db demo/data/ecom.db-* demo/data/ecom_offline.db demo/data/ecom_offline.db-*
```

và bật lại; cơ sở dữ liệu được tạo lại từ `demo/db/ecom_schema_sqlite.sql` và `demo/db/seed.sql`. **Đừng** gõ `ecom.db*`: nó xóa luôn file sao lưu `ecom.db.before-category`.

2.5 Năm phút trước

- ☐ Ba tab Chrome mở, showcase ở đầu mục A; cửa hàng ở trang chủ với C001; Shift+D đã tắt.
- ☐ VP mở sẵn các tab sơ đồ; VS Code mở sẵn các file; cửa sổ cơ sở dữ liệu ở bảng `category`.
- ☐ Đã quyết: có Gemma hay trình bày dự phòng (mục 2.3). Ảnh demo trên Desktop; micro đã cho phép.
- ☐ Đóng Zalo, Messenger, email; đóng file `app.local.properties` (có key). Điện thoại im lặng; nước uống.

3. Kịch bản 5–7 phút

Bảy bước, theo đúng thứ tự: vấn đề, sơ đồ, phân tích sang thiết kế, cơ sở dữ liệu, đặc tả sang code, demo, kiểm thử. Mỗi hộp ghi: **mở** cửa sổ nào, **nói** gì (hai đến bốn câu, nói lại bằng lời của mình cũng được), và **chỉ vào** đúng một chỗ. *[Chữ xám trong ngoặc]* là thao tác, không đọc lên. Phần "Nhớ thêm" là tên kỹ thuật, chỉ dùng khi thầy hỏi.

- **Nói trước, mở sau:** "bây giờ em mở sơ đồ hoạt động", rồi mới Cmd+Tab. Nhìn thầy, không nhìn màn hình.
- **Không đọc tên lớp, tên hàm.** Nói ý ("đơn hàng tự quyết có được hủy không"), còn tên thì để màn hình hiện.
- **Một cửa sổ không mở được** thì đừng loay hoay: showcase có ảnh của mọi sơ đồ và mọi màn hình.

Bước 1 · Vấn đề và yêu cầu ≈ 45 giây

Mở: Showcase, mục **A** (Project overview): năm thẻ vấn đề P1–P5, rồi kéo xuống "Functional scope".

Nói: Họa. là một cửa hàng nhỏ bán bản in tranh danh họa và khung tranh. Trước đây cửa hàng bán qua tin nhắn và bảng tính, nên có năm chỗ đau: khách không biết tên tranh thì không tìm được, tranh hết hàng vẫn bị bán, khách không biết đơn đang ở đâu, giá và đơn làm bằng tay, và dữ liệu của khách dễ bị lộ. Từ năm chỗ đau đó em viết ra 20 yêu cầu chức năng và 12 yêu cầu chất lượng, gom thành 14 việc mà khách và người quản lý cần làm. AI chỉ được dùng cho chỗ đau đầu tiên: giúp khách tìm tranh bằng lời, bằng giọng nói và bằng ảnh.

Chỉ vào: thẻ **P2 "Sold-out prints are sold"**: chỗ đau này sẽ đi theo suốt bài, tới tận câu lệnh trừ kho ở bước 4 và bước 6.

Nhớ thêm (không đọc lên):

- 20 FR (12 Must, 6 Should, 2 Could), 12 NFR, 14 use case, 4 tác nhân; viết trong `tools/model.py`.
- P2 → FR07, NFR11 → UC07; showcase B tô sẵn chuỗi này. Cây vấn đề: bài §1.6 (ảnh chụp bản dùng thử).

Bước 2 · Các sơ đồ chính ≈ 1 phút

Mở: Showcase, mục **B** (Development process): năm cột, chuỗi UC07 Checkout được tô ở mỗi cột. Rồi VP: "1.2 Use Case Diagram - Hoa Gallery", sau đó "2.2 Activity - Checkout".

Nói: Em làm đúng năm bước của môn, và lấy một việc là đặt hàng để theo suốt cả năm bước. *[chuyển sang VP, sơ đồ use case]* Ở bước yêu cầu, sơ đồ này cho biết ai làm gì: khách tìm tranh, đặt hàng, hủy đơn; người quản lý sửa giá và chuyển trạng thái đơn. *[mở sơ đồ hoạt động đặt hàng]* Ở bước phân tích, sơ đồ hoạt động kể từng bước của việc đặt hàng, có cả những ngã rẽ như giỏ trống hay hết hàng. Mỗi ngã rẽ là một luật của cửa hàng, và về sau mỗi luật thành một dòng code có bài kiểm thử riêng.

Chỉ vào: ô quyết định có hai nhánh `in stock` và `out of stock` trong sơ đồ hoạt động.

Nhớ thêm (không đọc lên):

- Project có 40 sơ đồ; 5 sơ đồ (2 ArchiMate, 1 BPMN, 2 DFD) là ảnh chụp bản dùng thử (câu hỏi ở mục 5.9).
- UC07 «include» UC06; luật BR02 (hết hàng), BR03 (giỏ trống); ba làn Customer / Họa. system / Database. Cùng kịch bản ở dạng tuần tự phân tích: "2.6b Analysis Sequence - Checkout (UC07)".

Bước 3 · Từ phân tích sang thiết kế ≈ 45 giây

Mở: Showcase, mục **C**, chuỗi 1 "Classes: analysis → design → code" (lớp Order ở bốn mức). Trong VP: "2.8 Domain Model - analysis class diagram", rồi "4.3 Class Diagram in Design", rồi "4.4a Three Layers - classes and their links".

Nói: Đây là phần em thấy quan trọng nhất của môn. Lúc phân tích, lớp đơn hàng chỉ có tên các thông tin và loại quan hệ. Sang thiết kế, nó có kiểu dữ liệu, có chiều quan hệ, và có những việc nó tự làm: tự tính tổng tiền, tự quyết có được hủy hay không, tự kiểm tra có phải đơn của khách này không. Rồi em xếp các lớp vào ba tầng theo bốn bước trong phụ lục của thầy; mũi tên chỉ đi xuống, nên phần giao diện không bao giờ chạm thẳng vào cơ sở dữ liệu.

Chỉ vào: trong "4.3 Class Diagram in Design", ba phương thức của `Order`: `cancel()`, `advanceTo()`, `isOwnedBy()`.

Nhớ thêm (không đọc lên):

- Bốn bước của phụ lục "Từ phân tích đến thiết kế" (bài §4.4.1): thêm phương thức; đổi liên kết thành «use»; DAO = interface + bản SQLite (`CartRepository` / `SqliteCartRepository` là `CartDAO` / `CartDAOImpl`); ba tầng có package.
- 18 lớp phân tích → 60 lớp Java và 9 view; 14 package. Lớp domain dùng chung cho cả ba tầng, không phải một tầng.

Bước 4 · Cơ sở dữ liệu ≈ 50 giây

Mở: Showcase, mục **E**, phần "Products, their category and the detail tables". Rồi VP "4.2 Database - entity relationship diagram". Rồi cửa sổ cơ sở dữ liệu, bảng `category`.

Nói: Theo góp ý của thầy, sản phẩm được xếp theo danh mục. Bảng sản phẩm chỉ giữ phần chung: tên, giá, số hàng trong kho; phần riêng nằm ở bảng chi tiết: tranh có họa sĩ, kích thước, chất liệu, còn khung có vật liệu và màu; và mỗi danh mục ghi rõ chi tiết của nó nằm ở bảng nào. Nhờ vậy giá và số hàng chỉ có một chỗ giữ, giỏ hàng và đơn hàng chỉ cần trỏ tới một chỗ. Cơ sở dữ liệu này không gõ tay: Visual Paradigm sinh câu lệnh tạo bảng từ mô hình, và em đã so nó với cơ sở dữ liệu đang chạy.

Chỉ vào: hai dòng của bảng `category`: **Fine-art prints** có `detail_kind` = PAINTING (bảng `painting`), **Frames** có FRAME (bảng `frame`).

Nhớ thêm (không đọc lên):

- 18 bảng, 19 khóa ngoại; `product.category_id` → `category` (1 – 0..*); `painting`, `frame` dùng khóa chính đồng thời là khóa ngoại tới `product`.
- Trình hướng dẫn *Generate Database* của VP (ảnh ở bài §4.2.3 và showcase E) viết `demo/db/vp_wizard_ddl.sql`; `build/schema_comparison.txt`: giống về bảng, 91 cột, kiểu, NOT NULL, khóa chính, UNIQUE và 19 khóa ngoại; chỉ thiếu 20 ràng buộc CHECK (sơ đồ không chứa chúng), `tools/gen_sql.py` thêm từ cùng mô hình. Kiểm tra SQL: 6/6 câu sai bị từ chối, 0 sản phẩm nằm sai bảng chi tiết.
- Thầy muốn xem VP sinh cơ sở dữ liệu tận nơi: mục 6.7 (một lệnh, khoảng 2 phút).

Bước 5 · Từ đặc tả đến code ≈ 45 giây

Mở: Showcase, mục **D**, thẻ "Cancel Order" (UC11). Rồi VS Code: `tools/model.py` dòng 359 (lớp `Order`) đặt cạnh `Order.java` dòng 41.

Nói: Mọi tên gọi trong bài, từ yêu cầu, lớp, phương thức đến bảng và cột, em chỉ viết một lần trong một file đặc tả. Từ file đó em sinh ra sơ đồ, câu lệnh tạo bảng và khung code. Lấy việc hủy đơn làm ví dụ: trên trang này em đi được từ yêu cầu, qua sơ đồ, tới đúng dòng code và đúng bài kiểm thử. Và có một công cụ đọc lại code thật để so với đặc tả: nếu em đổi tên một phương thức mà quên sửa thiết kế thì nó báo ngay.

Chỉ vào: hàng "4 · Implementation" của thẻ: liên kết `Order.cancel` tới `Order.java`, dòng 41.

Nhớ thêm (không đọc lên):

- Lần so gần nhất: 60 lớp, 169 trường, 185 phương thức, 26 hàm view, 18 bảng, 91 cột, 62 câu SQL: khớp hết.
- Công cụ sinh: `tools/build_vp.py` (sơ đồ), `tools/gen_sql.py` (bảng, dữ liệu mẫu), `tools/gen_java.py` (khung code ở `demo/generated`); công cụ so: `tools/check_code_model.py`.

Bước 6 · Demo trực tiếp ≈ 1 phút 45 giây

Mở: Tab cửa hàng (C001). Có Gemma: một phần Flow 1, Flow 3 và Flow 4 của mục 6. Không có Gemma: theo mục 6.6.

Nói: Bây giờ em cho thầy xem nó chạy thật. *[gõ blue landscape paintings]* Em gõ một câu bình thường, tranh phong cảnh màu xanh, và mỗi bức hiện ra kèm một dòng lý do vì sao được chọn. *[mở một bức, thêm vào giỏ, đặt hàng]* Dòng chữ nhỏ trên tên tranh là danh mục của nó, chính là dòng trong bảng lúc nãy; em đặt mua, và tổng tiền đã trừ mười phần trăm cho khách VIP, cộng tám đô phí giao. *[My orders, tra đơn 20261003]* Cuối cùng em thử xem đơn của một khách khác: cửa hàng trả lời như thể đơn đó không hề tồn tại.

Chỉ vào: trên trang chi tiết tranh, nhãn nhỏ **"Fine-art prints ▪ …"** phía trên tên tranh.

Nhớ thêm (không đọc lên):

- Tổng = tạm tính − 10% + 8 USD; giỏ có sẵn Woman with a Parasol (89 USD).
- Thầy muốn xem AI nhiều hơn: Flow 2 (ảnh, treo tranh lên tường phòng).

Bước 7 · Kiểm thử và kết luận ≈ 30 giây

Mở: Showcase, mục **G** (Verification & conclusion): hàng ô kết quả trên cùng, rồi phần "Limitations".

Nói: Để biết mình làm đúng, em có 67 bài kiểm thử tự động, một lần chạy thử từ đầu đến cuối trên cơ sở dữ liệu mới, và công cụ so code với đặc tả; tất cả đều đạt. Em cũng ghi rõ những gì chưa làm: đặt hàng chưa gói trong một giao dịch, hủy đơn chưa trả hàng về kho, và thanh toán chỉ là mô phỏng. Em xin hết phần trình bày, mời thầy đặt câu hỏi.

Chỉ vào: ô "tests pass" và ô "code = model".

Nhớ thêm (không đọc lên):

- 67 bài kiểm thử (lần chạy 2026-11-12 16:15), chạy thử đầu cuối 17/17 bước, có và không có Gemma (`build/e2e_results.txt`).
- Không phải yêu cầu nào cũng có kiểm thử tự động; bảng yêu cầu ở mục G ghi rõ yêu cầu nào có, yêu cầu nào chứng minh bằng demo hoặc trong bài.

3.1 Bản 3 phút (khi thầy cho ít thời gian)

Giữ đúng thứ tự, mỗi bước một câu, chỉ mở showcase (trừ demo).

Bước	Mở	Một câu
1 · 20 giây	showcase A	Năm chỗ đau của một cửa hàng tranh nhỏ thành 32 yêu cầu và 14 việc cần làm.
2+3 · 40 giây	showcase B, rồi C	Em theo việc đặt hàng qua năm bước; lớp đơn hàng đi từ vài thông tin lúc phân tích tới những việc nó tự làm lúc thiết kế, rồi được xếp vào ba tầng.
4 · 30 giây	showcase E	Danh mục, bảng sản phẩm chung và bảng chi tiết; Visual Paradigm sinh câu lệnh tạo bảng và em đã so khớp với cơ sở dữ liệu thật.
5 · 20 giây	showcase D	Một file đặc tả cho mọi tên gọi; từ yêu cầu em bấm tới đúng dòng code và đúng bài kiểm thử.
6 · 1 phút	cửa hàng	Tìm `blue landscape paintings`, rồi tra đơn 20261003 của khách khác: "không tìm thấy".
7 · 20 giây	showcase G	Kiểm thử và chạy thử đều đạt; ba hạn chế em đã ghi rõ.

3.2 Bản 10 phút, theo năm điểm đánh giá của thầy

Thầy chấm theo năm điểm: (1) biểu đồ lớp phân tích có kế thừa, (2) data model ⇒ DB, (3) class design: lớp phân tích + phương thức, (4) layer → sinh code, (5) code + demo. Showcase có mục **Evaluation 1–5** đi đúng thứ tự này: mở mục đó, cuộn từng bước, mỗi bước bấm vào hình để phóng to. Nói khoảng **6 phút 30**, để **3 phút 30** cho thầy hỏi. Trước mỗi bước nói số bước: *"Điểm thứ hai thầy chấm là data model sang cơ sở dữ liệu…"*

Phút	Điểm của thầy	Mở	Nói và chỉ vào
0:00–0:30	Mở đầu	showcase, mục Evaluation	Họa. bán bản in tranh; năm chỗ đau thành 20 yêu cầu chức năng, 12 yêu cầu chất lượng, 14 use case. Em trình bày theo đúng năm điểm thầy chấm.
0:30–1:30	**1**. Lớp phân tích có kế thừa	Evaluation bước 1 (bấm hình để phóng to); VP "2.8 Domain Model - analysis class diagram"	18 lớp, chỉ có thông tin và *loại* quan hệ, chưa có kiểu, chưa có phương thức. Ba cây kế thừa: khách hàng thường, VIP và mới; sản phẩm là tranh hoặc khung; thanh toán tiền mặt hoặc thẻ. Sản phẩm và thanh toán là lớp trừu tượng.
1:30–3:00	**2**. Data model ⇒ DB	Evaluation bước 2; VP "2.9b Logical Data Model (ORM-Persistable)", "4.2 Database - entity relationship diagram"	Khái niệm, rồi logic có tên quan hệ, bội số, «ORM-Persistable», rồi vật lý 18 bảng. Kế thừa thành bảng con dùng chung khóa với bảng cha. Theo góp ý của thầy: `category` phân loại, `product` giữ phần chung. Visual Paradigm tự sinh cơ sở dữ liệu; so với cơ sở dữ liệu đang chạy: giống 18 bảng và 19 khóa ngoại.
3:00–4:00	**3**. Class design: lớp + phương thức	Evaluation bước 3; VP "4.3 Class Diagram in Design"	Cùng các lớp phân tích, giờ có kiểu dữ liệu và phương thức; bảng trong bước 3 so số thuộc tính và liệt kê phương thức từng lớp. Kế thừa thành đa hình: VIP tự tính giảm giá, tranh và khung tự cho biết loại chi tiết. Cơ sở dữ liệu được sửa theo thiết kế: bảng *Database (revised)*.
4:00–5:00	**4**. Layer → sinh code	Evaluation bước 4; VP "4.4a Three Layers - classes and their links"	Bốn bước trong phụ lục của thầy: thêm phương thức, liên kết thành «use», DAO là interface cộng bản SQLite, ba tầng có package. Công cụ sinh một file Java cho mỗi lớp thiết kế, đúng package của tầng; lớp miền sinh ra là chạy được ngay, các lớp khác em viết thêm thân hàm.
5:00–6:30	**5**. Code + demo	Evaluation bước 5, rồi mục D (bảng "Every design class in the code"), rồi tab cửa hàng (C001)	Mọi lớp thiết kế có trong code, đúng từng trường, từng phương thức, đúng chiều giữa các tầng; code dùng SQLite thật. Demo: gõ `blue landscape paintings`, mở một bức (dòng danh mục), tra đơn 20261003 của khách khác: "không tìm thấy".
6:30	Kết	Evaluation bước 5	67 bài kiểm thử, chạy thử đầu cuối 17/17 bước đều đạt; ba hạn chế đã ghi. Mời thầy hỏi.

- **Nếu 10 phút là chỉ để trình bày** (hỏi đáp tính riêng): thêm 1 phút 30 cho chương 3 sau phần demo: tìm bằng ảnh và trợ lý tra đơn bằng công cụ, rồi nói vì sao AI chỉ gợi ý, không quyết định.
- **Bị cắt ngang**: trả lời ngắn rồi nhảy tới bước kế tiếp bằng các nút *1 · … 5* ở đầu mục Evaluation.
- **Còn 2 phút mà mới tới bước 3**: nói bước 3 và 4 mỗi bước một câu, bỏ demo, giữ phần kết.

4. Quyết định thiết kế

Mười quyết định thầy dễ hỏi nhất. Mỗi hộp trả lời năm câu: **vì sao** chọn, **phương án khác** là gì và vì sao không chọn, giải quyết **yêu cầu** nào, nằm **ở đâu trong code**, và **nếu yêu cầu đổi** thì sửa gì. Khi trả lời, nói hai dòng đầu bằng lời thường; mở dòng "Trong code" trên VS Code nếu thầy muốn xem. Đường dẫn Java tính từ `demo/src/main/java/com/ptit/ecom/`.

QĐ1 · Danh mục + bảng sản phẩm chung + bảng chi tiết

Vì sao? Cửa hàng bán hai loại hàng có thông tin khác nhau: tranh có họa sĩ, phong cách, năm, kích thước; khung có vật liệu, màu, cỡ. Thầy góp ý không để một bảng sản phẩm ôm hết. Em tách: danh mục phân loại sản phẩm và ghi chi tiết nằm ở bảng nào; bảng sản phẩm giữ phần chung (tên, giá, kho); bảng tranh và bảng khung giữ phần riêng. Giá và kho chỉ có một chỗ, nên giỏ, đơn và luật trừ kho dùng chung cho mọi loại hàng.

Phương án khác Một bảng rộng có mọi cột: phần lớn ô bỏ trống, và một cái khung vẫn "có" họa sĩ. Mỗi loại một bảng riêng có giá và kho riêng: lặp cột chung, và dòng giỏ, dòng đơn không thể có một khóa ngoại tới "sản phẩm". Danh mục chỉ là một cột chữ: không ai biết chi tiết nằm ở bảng nào.

Yêu cầu Góp ý của thầy; FR19 (mỗi sản phẩm thuộc đúng một danh mục), NFR11 (toàn vẹn dữ liệu), FR05, FR12.

Trong code `TABLES` trong `tools/model.py`; `ecom_schema_sqlite.sql` dòng 48 (`category`), 57 (`product`), 79 (`painting`), 95 (`frame`); `Category.accepts` (`domain/product/Category.java`:13); `SqliteProductRepository.toProduct` (dòng 97, đọc bảng chi tiết theo `detailKind`) và `insertProduct` (dòng 202, từ chối sản phẩm sai danh mục). Kiểm thử: `RepositoryTest.java`:57, `DomainTest.java`:107.

Nếu yêu cầu đổi Thêm một dòng hàng có cùng thông tin (ví dụ tranh in canvas): thêm một dòng vào bảng `category`, không sửa code. Thêm loại hàng có thông tin mới (ví dụ tượng): thêm một bảng chi tiết, một giá trị `DetailKind` và hàm đọc, ghi của nó; giỏ, đơn và kho không đổi.

QĐ2 · Món trong giỏ khác món trong đơn; đặt hàng là một phép biến đổi

Vì sao? Giỏ là ý định tạm thời: giá có thể đổi, khách có thể bỏ bớt. Đơn là lịch sử: giá đã chốt, không bao giờ đổi. Lúc đặt hàng, cửa hàng đọc lại giá và kho hiện tại rồi tạo món trong đơn với giá chốt, sau đó giỏ trống. Vì giỏ không "sở hữu" đơn lâu dài, quan hệ giữa hai thứ là một phụ thuộc «create», không phải liên kết.

Phương án khác Một lớp "món hàng" có cờ trạng thái: trộn hai vòng đời, và đơn cũ sẽ đổi tiền khi danh mục đổi giá. Lấy luôn giá trong giỏ: khách có thể trả giá cũ đã hết hiệu lực. Nối đơn với giỏ bằng liên kết: giỏ bị làm rỗng ngay sau đó nên liên kết vô nghĩa.

Yêu cầu FR07; luật BR04 (giá trên đơn là giá lúc đặt), BR03 (giỏ trống không đặt được), BR02 (hết hàng); FR15, FR17 (giảm giá VIP, phí giao).

Trong code `OrderItem.fromCartItem` (`domain/order/OrderItem.java`:16); `CheckoutService.checkout` (`application/service/CheckoutService.java`:45) và `currentProduct` (dòng 100); cột `cart_item.cart_price` khác `order_item.unit_price`. Kiểm thử `DomainTest.java`:63. Sơ đồ "2.8 Domain Model - analysis class diagram" (mũi tên «create»).

Nếu yêu cầu đổi Muốn giữ giá trong giỏ 24 giờ: chỉ sửa chỗ đọc giá trong `currentProduct` và `fromCartItem`. Muốn đặt một phần giỏ: vòng lặp chỉ lấy những dòng được chọn; lớp đơn không đổi.

QĐ3 · Ba tầng, mô hình miền dùng chung; Repository chính là DAO

Vì sao? Mỗi phần lo một việc: giao diện nhận yêu cầu, tầng ứng dụng điều phối, tầng dữ liệu cất và lấy dữ liệu, gọi AI. Mũi tên chỉ đi xuống. Em đi đúng bốn bước trong phụ lục "Từ phân tích đến thiết kế" của thầy: thêm phương thức, đổi liên kết thành «use», dùng DAO (interface + bản cài đặt), chia ba tầng có package. Vì tầng trên chỉ biết interface, có thể thay cơ sở dữ liệu hoặc dùng bản giả khi kiểm thử.

Phương án khác Controller gọi SQL trực tiếp: ít file hơn nhưng giao diện chạm vào dữ liệu, vi phạm NFR04. Đối tượng tự lưu mình (active record): trộn luật nghiệp vụ với chuyện lưu trữ. Vẽ miền thành tầng thứ tư như bài giảng 05: cùng lớp, cùng package, chỉ khác cách vẽ; em chọn ba tầng như bài tập A06 và bài mẫu.

Yêu cầu NFR04 (giao diện không chạm cơ sở dữ liệu, AI sau interface), NFR10, NFR06.

Trong code `LAYERS` trong `tools/model.py`:742; `ProductRepository` (interface) và `SqliteProductRepository`; mọi đối tượng được tạo một lần trong `EcomApplication.main` (`presentation/web/EcomApplication.java`:40). Công cụ kiểm tra import từng file: `check_dependencies` (`tools/check_code_model.py`:298). VP: "4.4a Three Layers - classes and their

	links", "4.4c Package Diagram - Design Packages", "4.4a Three Logical Components".
Nếu yêu cầu đổi	Đổi sang PostgreSQL: viết bốn lớp repository mới và sửa vài dòng trong `EcomApplication.main`; tầng ứng dụng không đổi. Thêm ứng dụng di động: thêm phần giao diện mới gọi cùng các service.

QĐ4 · Máy trạng thái của đơn nằm trong chính lớp Order

Vì sao?	Luật "được hủy khi nào" và "được chuyển sang trạng thái nào" thuộc về đơn hàng, nên đặt ở một chỗ duy nhất là lớp đơn. Khách hủy, quản trị viên chuyển trạng thái hay trợ lý tra đơn đều phải đi qua chỗ đó, không ai đi vòng được.
Phương án khác	Viết `if` rải rác trong các service và controller: luật bị lặp, dễ quên một chỗ. Đặt luật ở giao diện (ẩn nút hủy): gọi thẳng API là vượt qua. Đặt trong cơ sở dữ liệu: ràng buộc CHECK chỉ kiểm được giá trị hợp lệ, không kiểm được bước chuyển, và khó kiểm thử.
Yêu cầu	FR11 (hủy khi chưa giao), FR13 (chuyển trạng thái), BR06, BR07.
Trong code	`Order.cancel` và `Order.advanceTo` (`domain/order/Order.java`:41, 47); `OrderService.cancelOrder` (40) và `updateStatus` (51) chỉ tải đơn, gọi luật, lưu lại. CHECK giá trị trạng thái ở bảng `orders` (dòng 136). Kiểm thử `DomainTest.java`:78, 85. VP: "2.4 State Machine - Order".
Nếu yêu cầu đổi	Cho trả hàng sau khi giao: thêm trạng thái RETURNED vào `OrderStatus`, vào danh sách CHECK trong `TABLES`, và một bước chuyển trong `advanceTo`; cập nhật sơ đồ trạng thái và một bài kiểm thử.

QĐ5 · Đơn của người khác trả lời là "không tìm thấy"

Vì sao?	Mã khách luôn lấy từ phiên đăng nhập, không bao giờ lấy từ những gì trình duyệt gửi lên. Đơn của người khác nhận đúng câu trả lời như đơn không tồn tại, nên không ai dò được mã đơn nào là thật. Trợ lý AI cũng chạy công cụ tra đơn với mã khách của phiên, và các công cụ không có tham số mã khách.
Phương án khác	Trả lời "đơn này không phải của bạn" (mã 403): lộ ra rằng đơn tồn tại. Chỉ kiểm tra ở giao diện: gọi thẳng API là qua. Nhận mã khách trong yêu cầu: ai cũng giả được.
Yêu cầu	NFR01, FR09, luật BR08.
Trong code	`OrderService.getOrder` (`application/service/OrderService.java`:21) gọi `Order.isOwnedBy` (`Order.java`:55); `WebServer.customer` (`presentation/web/WebServer.java`:256) lấy mã khách từ phiên; lỗi "không tìm thấy" thành HTTP 404 (dòng 105); `AssistantService.runTool` (dòng 77). Kiểm thử `OrderServiceTest.java`:27; bước chạy thử "UC08/NFR01".
Nếu yêu cầu đổi	Thêm đăng nhập có mật khẩu: chỉ đổi phần phiên (`SessionManager`). Cho nhân viên chăm sóc khách xem mọi đơn: thêm một vai trò và một đường riêng như trang quản trị, không nới luật của khách.

QĐ6 · AI đứng sau interface và luôn có đường lui

Vì sao?	Gemma ở máy chủ xa, có thể chậm hoặc hỏng; tối 11/11 máy chủ của lớp đã trả lỗi 502 thật. AI chỉ giúp *hiểu* (câu, ảnh, phòng, câu hỏi); giá, kho, quyền xem đơn do luật của cửa hàng quyết. Mỗi tính năng AI có một cách làm không cần AI, và mỗi lần gọi chờ tối đa 15 giây.
Phương án khác	Gọi Gemma thẳng từ service: không có đường lui, không kiểm thử được khi thiếu máy chủ. Tìm kiếm chỉ bằng AI: máy chủ hỏng là cửa hàng đứng. Không dùng AI: không giải được chỗ đau P1 (khách không biết tên tranh).
Yêu cầu	NFR02, NFR12, NFR04, NFR07; luật BR11 (gợi ý của AI không đổi giá, kho, quyền).
Trong code	`LlmClient` (`infrastructure/ai/LlmClient.java`:11), `GemmaClient`; chữ: `QueryService.understand` (73) → `parseRules` (93); ảnh: `ImageService.describe` (69) → `readColorLocally` (117); trợ lý: `AssistantService.ask` (43) → `answerWithoutAi` (103). Kiểm thử `SearchTest.java`:76. Thời gian chờ: `gemma.timeoutSeconds=15` trong `demo/config/app.properties`.
Nếu yêu cầu đổi	Đổi nhà cung cấp mô hình: **đã xảy ra thật** tối 11/11, chuyển từ máy chủ của lớp sang Gemma 4 trên DeepInfra chỉ bằng hai dòng cấu hình và key, không sửa dòng code nào. Mô hình khác API: viết một lớp mới cài `LlmClient`, sửa một dòng trong

	`EcomApplication.main`.

QĐ7 · Whisper tự chạy trên máy

Vì sao?	Giọng nói là dữ liệu riêng tư: một câu nói trong phòng khách có thể có tiếng người khác. Whisper chạy ngay trên laptop nên âm thanh không rời máy, không tốn tiền theo phút, và đủ nhanh: đo được trung bình 0,70 giây, lâu nhất 1,43 giây cho một câu, trong khi yêu cầu là 3 giây. Khách thấy chữ và sửa được trước khi tìm.
Phương án khác	Nhận dạng giọng nói có sẵn của trình duyệt: thường gửi âm thanh lên máy chủ của hãng trình duyệt, không điều khiển được. Dịch vụ nhận dạng trên mây: âm thanh ra ngoài, cần mạng, tốn tiền.
Yêu cầu	NFR08 (âm thanh chỉ xử lý trên máy, câu 5 giây nghe xong trong 3 giây), FR02; NFR06.
Trong code	`demo/voice-server/server.py` (cổng 9000, `demo/run.sh` tự bật); `WhisperClient.transcribe` (`infrastructure/ai/WhisperClient.java`:32) sau interface `SpeechClient`; `SpeechService.transcribe` (`application/search/SpeechService.java`:17). Whisper tắt thì giao diện mời khách gõ chữ.
Nếu yêu cầu đổi	Cần tiếng Việt: đổi tham số ngôn ngữ và dùng mô hình lớn hơn (tiếng Việt hiện kém hơn nhiều). Nhiều khách cùng lúc: chuyển máy chủ giọng nói sang một máy riêng của cửa hàng, vẫn không gửi ra ngoài.

QĐ8 · Một bản đặc tả duy nhất sinh ra sơ đồ, bảng và khung code, kèm công cụ so khớp

Vì sao?	Khi một tên được gõ ở ba nơi (sơ đồ, cơ sở dữ liệu, code) thì sớm muộn sẽ lệch. Em viết mọi yêu cầu, use case, lớp, phương thức, bảng và cột đúng một lần, rồi sinh mọi thứ từ đó. Công cụ so khớp đọc lại code thật, giao diện và cơ sở dữ liệu thật mỗi lần kiểm thử, nên "code khớp thiết kế" là điều được kiểm tra, không phải lời hứa.
Phương án khác	Vẽ tay trong VP rồi viết code theo trí nhớ: không gì báo khi lệch. Sinh khung code một lần rồi bỏ mặc: sau vài lần sửa là lệch. Chỉ so bằng mắt: không lặp lại được.
Yêu cầu	NFR10 (tên trong code trùng mô hình, kiểm tra tự động), NFR11.
Trong code	`tools/model.py` (`REQUIREMENTS` dòng 18, `ANALYSIS` 141, `DESIGN` 235, `TABLES` 773); sinh: `tools/build_vp.py`, `tools/gen_sql.py`, `tools/gen_java.py` (khung ở `demo/generated`); so: `tools/check_code_model.py`, chạy trong `demo/test.sh`; kết quả ở cuối `build/test_results.txt`.
Nếu yêu cầu đổi	Thêm một cột hoặc một phương thức: sửa `tools/model.py`, dựng lại sơ đồ, bảng và bảng biểu trong bài; công cụ so khớp báo lỗi cho tới khi code theo kịp.

QĐ9 · SQLite + JDBC, không framework; trừ kho bằng một câu UPDATE có điều kiện

Vì sao?	Bám code mẫu A05, A06 của môn: các tầng hiện rõ trong code, không có "phép màu" sinh ngầm. SQLite chỉ là một file nên chạy trên một máy, không cần cài máy chủ. Trừ kho bằng một câu lệnh vừa kiểm tra vừa trừ: chỉ trừ khi còn đủ hàng. Hai người mua cùng lúc món cuối cùng thì chỉ một người thành công, kho không bao giờ âm; ràng buộc CHECK của bảng là chốt chặn cuối.
Phương án khác	Spring Boot và một ORM: nhiều phần ẩn, tên bảng có thể do framework đặt, nặng cho bài môn học. Đọc kho, so trong Java rồi ghi lại: giữa lúc đọc và lúc ghi, người khác có thể mua mất. MySQL hay PostgreSQL: phải cài và chạy máy chủ, không còn "một máy, chạy ngoại tuyến".
Yêu cầu	NFR06, NFR11 (kho không âm), FR07, BR02, NFR10 (mọi câu SQL được so với mô hình).
Trong code	`SqliteProductRepository.decreaseStock` (`infrastructure/repository/sqlite/SqliteProductRepository.java`:70): `UPDATE product SET stock_quantity = stock_quantity - ? WHERE product_id = ? AND stock_quantity >= ?`; CHECK ở `ecom_schema_sqlite.sql`:62; bật khóa ngoại cho mỗi kết nối (`Database.java`:25). Kiểm thử `RepositoryTest.java`:82.
Nếu yêu cầu đổi	Nhiều người dùng cùng lúc: thay bằng cơ sở dữ liệu dạng máy chủ qua interface repository. Muốn đặt hàng "tất cả hoặc không gì": gói lưu đơn, trừ kho, làm rỗng giỏ vào một giao dịch (hạn chế đã biết).

QĐ10 · Cơ sở dữ liệu sinh từ mô hình vật lý trong VP, rồi so với cơ sở dữ liệu thật

Vì sao?	Thầy yêu cầu đi từ mô hình dữ liệu tới "generate database". Trình hướng dẫn Generate Database của chính Visual Paradigm đọc các bảng trong sơ đồ quan hệ thực thể và viết câu lệnh tạo bảng; em nạp nó và cơ sở dữ liệu đang chạy vào SQLite rồi so từng bảng. Kết quả giống nhau về bảng, cột, kiểu, NOT NULL, khóa chính, UNIQUE và cả khóa ngoại. Chỉ các ràng buộc kiểm tra giá trị (CHECK) là sơ đồ không chứa, nên công cụ của em thêm vào từ cùng mô hình. Lần chạy đầu VP còn bắt được lỗi của em: sáu khóa ngoại cùng tên "is a"; em đặt tên theo quy ước `fk_<table>_<column>` (khóa ngoại, tên bảng, tên cột).
Phương án khác	Viết SQL bằng tay: lệch với sơ đồ. Chỉ dùng câu lệnh của VP: cơ sở dữ liệu sẽ không có khóa ngoại và CHECK, tức không từ chối dữ liệu sai. Chỉ dùng công cụ của em: thiếu bằng chứng từ chính VP.
Yêu cầu	Yêu cầu của thầy ở chương 4 (Generate Data Base); NFR11, NFR10.
Trong code	Trình hướng dẫn của VP viết `demo/db/vp_wizard_ddl.sql` (chạy bằng `tools/vp_generate_db.py`, ảnh `figures/vp/c42_generate_db.png`); mỗi lần dựng project, bộ sinh SQL của VP còn viết `demo/db/vp_generated_ddl.sql`; tên khóa ngoại: `fk_name` trong `tools/model.py`; `tools/compare_schema.py` → `build/schema_comparison.txt`; `tools/gen_sql.py` (`create_table` dòng 69, `validate` dòng 195) → `demo/db/ecom_schema_sqlite.sql`, `seed.sql`, `build/sql_validation.txt`; nạp lúc khởi động: `Database.initialize` (`Database.java`:34). Bài §4.2.3.
Nếu yêu cầu đổi	Thêm một cột: sửa mô hình, dựng lại cả hai bản và chạy lại phép so. Chuyển sang hệ cơ sở dữ liệu khác: đổi cấu hình cơ sở dữ liệu của project VP, sinh lại và so lại. Muốn thấy tận mắt: mục 6.7.

5. Câu hỏi phản biện

Mỗi câu có hai tầng: **Nói ngắn** là câu trả lời bằng lời thường, nói trước; **Nếu thầy hỏi tiếp** mới dùng tên kỹ thuật và mở file. Chưa chắc thì nói "em xin mở đúng chỗ để trả lời" và mở theo bảng ở mục 8.2; đừng đoán.

5.1 Phân tích và thiết kế

Hỏi: Phân tích và thiết kế khác nhau ở đâu?

Nói ngắn: Phân tích hỏi *cửa hàng cần gì*: khái niệm, thông tin, luật, nói bằng ngôn ngữ của người bán. Thiết kế hỏi *làm thế nào*: kiểu dữ liệu, phương thức, ai gọi ai, tầng nào, bảng nào. Ví dụ đơn hàng: lúc phân tích chỉ có ngày đặt, trạng thái, tổng tiền; lúc thiết kế thì tiền có kiểu số chính xác, trạng thái là một danh sách cố định, và đơn tự biết hủy, tự biết chuyển trạng thái.

Nếu thầy hỏi tiếp: "2.8 Domain Model - analysis class diagram" không có kiểu, quan hệ chỉ ghi loại; "4.3 Class Diagram in Design" có `BigDecimal`, `OrderStatus`, tên vai trò, phương thức. Tuần tự: "2.6b Analysis Sequence - Checkout (UC07)" so với "4.4d Checkout across the layers (design sequence)". Bảng chuyển đổi: bài §4.1.4.

Hỏi: Vì sao sơ đồ lớp phân tích không có phương thức, cũng không có bội số?

Nói ngắn: Theo quy tắc của thầy, mỗi sơ đồ thêm đúng một loại thông tin. Lớp phân tích chỉ có thuộc tính và loại quan hệ. Mô hình dữ liệu thêm tên quan hệ và số lượng ở hai đầu. Thiết kế mới thêm kiểu, phương thức và chiều quan hệ.

Nếu thầy hỏi tiếp: Showcase C, chuỗi 1, trình bày đúng ba bước này cho lớp `Order`, kèm "thay đổi gì, vì sao, được gì" ở mỗi bước.

Hỏi: Từ 18 lớp phân tích sao thành 60 lớp Java?

Nói ngắn: Các lớp miền giữ nguyên tên. Thiết kế thêm những lớp "làm việc": nhận yêu cầu web, điều phối từng việc, cất và lấy dữ liệu, gọi AI, cùng vài kiểu liệt kê như trạng thái đơn, trạng thái giao hàng.

Nếu thầy hỏi tiếp: Controller, service, interface repository và bản SQLite, `Database`, `GemmaClient`, `WhisperClient`; `DetailKind`, `OrderStatus`, `ShippingStatus`, `PaymentStatus`. Cộng 9 view JavaScript là 69 lớp thiết kế.

5.2 Mỗi loại sơ đồ để làm gì

Thầy chỉ vào một sơ đồ và hỏi “sơ đồ này để làm gì?”: trả lời bằng cột giữa, rồi chỉ vào ví dụ ở Họa.

Sơ đồ	Trả lời câu hỏi	Ở Họa. (tên trong VP)
Ngữ cảnh / DFD mức 0	Ranh giới: cái gì ở trong, ai ở ngoài, thông tin gì đi vào, đi ra	1.7 Context Diagram - DFD Level 0 (bản UML: 1.1 Context View)
DFD mức 1	Mở hộp: các tiến trình chính và kho dữ liệu, cân bằng với mức 0	2.9c Data Flow Diagram - Level 1
BPMN	Quy trình nghiệp vụ từ đặt hàng đến giao: ai làm gì, ở làn nào, ngoại lệ ra sao	1.4 Business Process (BPMN)
Bản đồ bên liên quan	Ai quan tâm điều gì, mục tiêu nào của hệ thống trả lời họ	1.3 Stakeholder Map (ArchiMate)
Cây vấn đề	Nguyên nhân gốc → vấn đề → tác động; từ đó ra nhu cầu	1.6 Problem Tree (ArchiMate)
Use case	Ai dùng hệ thống để đạt mục tiêu gì; include, extend, tổng quát hóa	1.2 Use Case Diagram; 2.4a–c theo nhóm
Hoạt động	Các bước của một việc và mọi ngã rẽ theo luật	2.2 Activity - Checkout (và 4 sơ đồ khác)
Tuần tự phân tích (BCE)	Đối tượng nào chịu trách nhiệm từng bước: trang, bộ điều khiển, thực thể	2.6a–c Analysis Sequence
Máy trạng thái	Vòng đời của đơn và bước chuyển hợp lệ	2.4 State Machine - Order
Lớp phân tích	Khái niệm miền, thuộc tính, loại quan hệ	2.8 Domain Model; 4.1 Class Diagram in Analysis
Dữ liệu mức khái niệm	Cần cất những thực thể nào, nối với nhau ra sao	2.9a Conceptual Data Model
Dữ liệu mức logic	Thuộc tính lưu trữ, tên quan hệ, bội số hai đầu	2.9b Logical Data Model
Dữ liệu mức vật lý	Bảng, cột, kiểu SQL, khóa chính, khóa ngoại, UNIQUE	4.2 Physical Data Model
ERD	Chính cơ sở dữ liệu, nguồn để VP sinh câu lệnh tạo bảng	4.2 Database - entity relationship diagram
Thành phần (component)	Ba khối logic và thứ đi qua mỗi phụ thuộc (HTTP, SQL, ảnh, âm thanh)	4.4a Three Logical Components
Lớp thiết kế	Kiểu, phương thức, chiều quan hệ, tên vai trò	4.3 Class Diagram in Design; 4.4a–c theo tầng
Tuần tự thiết kế	Lớp thật gọi phương thức thật qua các tầng, đúng như code chạy	4.4d Checkout across the layers; 4.4d, 4.4e

Hỏi: Sơ đồ ngữ cảnh và DFD mức 1 khác gì?

Nói ngắn: Sơ đồ ngữ cảnh coi cửa hàng là một hộp kín: chỉ thấy bốn bên ở ngoài và thông tin đi qua lại. DFD mức 1 mở cái hộp ra: bốn việc chính bên trong và ba nơi cất dữ liệu. Mọi dòng đi vào, đi ra ở sơ đồ ngữ cảnh phải có lại ở mức 1; đó gọi là cân bằng.

Nếu thầy hỏi tiếp: Bốn thực thể ngoài: Customer, Administrator, Payment Service, Gemma 4 Service; tám luồng có nhãn. Kho D1 Paintings and artists, D2 Carts, D3 Orders, payments, shipping. Whisper nằm trong ranh giới (chạy trên máy), Gemma và thanh toán ở ngoài.

Hỏi: Tuần tự phân tích và tuần tự thiết kế khác nhau thế nào? Còn sơ đồ hoạt động?

Nói ngắn: Sơ đồ hoạt động kể *việc gì xảy ra, theo thứ tự nào*. Tuần tự phân tích kể *ai chịu trách nhiệm*: trang mà khách thấy, một bộ điều khiển cho việc đó, và các thực thể giữ dữ liệu, chưa có tên lớp code. Tuần tự thiết kế dùng đúng lớp và phương thức trong code, đi qua ba tầng.

Nếu thầy hỏi tiếp: “2.6b Analysis Sequence - Checkout (UC07)”: `CheckoutUI` «boundary», `CheckoutControl` «control», `Cart`, `Painting`, `Order`, `OrderItem`, `Payment` «entity». “4.4d Checkout across the layers (design sequence)”: `CheckoutController` → `CheckoutService.checkout` → `OrderItem.fromCartItem` và các repository; mọi thao tác trong sơ đồ thiết kế đều có trong `DESIGN` và trong code.

5.3 UML và code có khớp nhau không

Hỏi: Làm sao em chứng minh code khớp sơ đồ?

Nói ngắn: Mọi tên chỉ viết một lần trong file đặc tả; sơ đồ, bảng và khung code được sinh từ đó. Rồi một công cụ đọc lại code thật, giao diện và cơ sở dữ liệu thật, so với đặc tả, và báo mọi thứ khớp. Nếu em đổi tên một phương thức mà không sửa đặc tả, lần kiểm thử tiếp theo báo lỗi ngay.

Nếu thầy hỏi tiếp: `tools/check_code_model.py`, chạy trong `demo/test.sh`: 60 lớp, 169 trường, 185 phương thức, 26 hàm view, 18 bảng, 91 cột, 62 câu SQL; kiểm cả chiều phụ thuộc giữa các tầng trên import. Dòng cuối `build/test_results.txt`.

Hỏi: Một mũi tên trên sơ đồ lớp thành gì trong code?

Nói ngắn: Mũi tên "dùng" thành một tham chiếu được truyền vào lúc tạo và một lời gọi. Mũi tên "hiện thực" thành `implements`. Liên kết có tên vai trò thành một trường. Quan hệ thành phần thành một danh sách bên trong, và trong bảng là khóa ghép, xóa cha thì xóa con. Kế thừa thành `extends`, và bảng con dùng chung khóa với bảng cha.

Nếu thầy hỏi tiếp: `CheckoutService` nhận `CartRepository...` trong hàm tạo; `SqliteOrderRepository implements OrderRepository`; `Order.customer`, `Order.items`; `order_item` có khóa (`order_id`, `product_id`) và `ON DELETE CASCADE`; `Painting extends Product`.

5.4 Mô hình dữ liệu, khóa và quan hệ

Hỏi: Ba mức của mô hình dữ liệu khác nhau thế nào?

Nói ngắn: Mỗi mức thêm chi tiết cho mức trước. Mức khái niệm chỉ có tên các thứ cần cất và các đường nối có tên. Mức logic thêm thông tin cần lưu và số lượng ở hai đầu mỗi đường nối. Mức vật lý thêm bảng, cột có kiểu, khóa chính, khóa ngoại; từ đó sinh ra cơ sở dữ liệu thật.

Nếu thầy hỏi tiếp: Logic: «ORM-Persistable», bội số 1, 0..1, 0..*, 1..*; kế thừa thành "is a" 1 – 0..1. Vật lý: «table», `numeric(12,2)`, {PK}, {FK}, {U}. Bài §2.10 (khái niệm, logic) và §4.2.1 (vật lý).

Hỏi: Khóa ngoại của danh mục hoạt động thế nào?

Nói ngắn: Mỗi sản phẩm trỏ tới đúng một danh mục và không được để trống; một danh mục có thể có nhiều sản phẩm hoặc chưa có cái nào. Cơ sở dữ liệu từ chối một sản phẩm trỏ tới danh mục không tồn tại. Còn luật "chi tiết phải nằm đúng bảng danh mục chỉ định" thì khóa ngoại không diễn tả được, nên em giữ nó trong code và kiểm tra lại bằng một truy vấn.

Nếu thầy hỏi tiếp: `product.category_id` `NOT NULL` → `category.category_id`, nhãn "classifies" 1 – 0..*; `category.detail_kind` có CHECK (PAINTING, FRAME). `Category.accepts`; `build/sql_validation.txt`: 0 sản phẩm nằm sai bảng chi tiết.

Hỏi: Vì sao khóa chính của bảng tranh cũng là khóa ngoại?

Nói ngắn: Vì một bức tranh *là một* sản phẩm: dòng trong bảng tranh và dòng trong bảng sản phẩm dùng chung một mã. Là khóa chính nên mỗi sản phẩm có nhiều nhất một dòng tranh; là khóa ngoại nên không có tranh nào thiếu phần chung.

Nếu thầy hỏi tiếp: Kế thừa "mỗi lớp một bảng": `painting`, `frame`, `customer_vip`, `customer_new`, `pay_cash`, `pay_credit`; quan hệ "is a" 1 – 0..1; `ON DELETE CASCADE` từ `product`.

Hỏi: Vì sao dòng giỏ và dòng đơn có khóa ghép?

Nói ngắn: Một dòng được nhận ra bằng cặp "giỏ nào, sản phẩm nào". Thêm cùng một bức hai lần thì số lượng tăng, không sinh hai dòng. Đây là quan hệ thành phần: xóa giỏ hay xóa đơn thì các dòng của nó đi theo.

Nếu thầy hỏi tiếp: `PRIMARY KEY (cart_id, product_id)` và `(order_id, product_id)`, `ON DELETE CASCADE` trên `cart_id`, `order_id`; `Cart.addItem` gộp dòng. Hệ quả: mỗi sản phẩm một dòng trong một đơn, số lượng ở cột `quantity` (CHECK > 0).

Hỏi: Vì sao mã khách trong bảng giỏ là duy nhất (UNIQUE)?

Nói ngắn: Vì mỗi khách chỉ có tối đa một giỏ đang dùng (luật BR01). Quan hệ "một khách có không hoặc một giỏ" được lưu bằng khóa ngoại cộng ràng buộc duy nhất; tạo giỏ thứ hai cho cùng một khách thì cơ sở dữ liệu từ chối.

Nếu thầy hỏi tiếp: `cart.customer_id NOT NULL UNIQUE`; cùng cách với `shipping.order_id` (đơn có không hoặc một lần giao). `build/sql_validation.txt`: "second cart for C001" bị từ chối.

Hỏi: Khóa ngoại giữ được "nhiều nhất", còn "ít nhất một" (đơn có ít nhất một dòng) thì sao?

Nói ngắn: Đúng, khóa ngoại và ràng buộc duy nhất chỉ giữ được cận trên. Cận dưới em giữ ở code: không cho đặt hàng khi giỏ trống, và không cho dùng địa chỉ của khách khác.

Nếu thầy hỏi tiếp: `CheckoutService.checkout` từ chối giỏ trống (BR03, lỗi 409); `Customer.findAddress`; bài §4.2.1, đoạn cuối.

5.5 Phân chia trách nhiệm

Hỏi: Ai quyết định một đơn có được hủy?

Nói ngắn: Chính đơn hàng. Phần điều phối chỉ làm bốn việc: tải đơn, kiểm tra đúng chủ, bảo đơn tự hủy, rồi lưu lại. Phần nhận yêu cầu web chỉ đổi yêu cầu thành một lời gọi. Mỗi lớp một việc, nên luật nằm đúng một chỗ và kiểm thử được mà không cần chạy web.

Nếu thầy hỏi tiếp: `OrderController.cancelOrder` → `OrderService.cancelOrder` (40) → `getOrder` + `Order.cancel` (41) → `OrderRepository.updateStatus`. QĐ4.

Hỏi: Vì sao không gọi Gemma ngay trong controller, hay trong lớp miền?

Nói ngắn: Vì AI là một dịch vụ bên ngoài, giống cơ sở dữ liệu: nó nằm ở tầng dữ liệu, sau một interface. Phần điều phối quyết định dùng câu trả lời thế nào và chuyển sang luật khi AI lỗi. Lớp miền không biết gì về AI, nên luật kinh doanh được kiểm thử mà không cần mạng.

Nếu thầy hỏi tiếp: `QueryService`, `ImageService`, `AssistantService` dùng `LlmClient`; `SpeechService` dùng `SpeechClient`. QĐ6.

5.6 Lựa chọn kiến trúc

Hỏi: Vì sao ba tầng mà không phải bốn? Tầng có phải là thư mục không?

Nói ngắn: Ba tầng như bài tập A06 và bài mẫu; các lớp miền như giỏ, đơn, khách là mô hình dùng chung cho cả ba tầng, không thuộc riêng tầng nào. Bài giảng 05 vẽ miền thành ô thứ tư; lớp và package vẫn y như vậy. Tầng là trách nhiệm, còn package là cách xếp code: mỗi tầng gồm nhiều package.

Nếu thầy hỏi tiếp: 3 tầng, 14 package (`presentation.*`, `application.*`, `infrastructure.*`, `domain.*`). Ngoại lệ duy nhất mà công cụ so khớp ghi chú: `EcomApplication` tạo các đối tượng của tầng dữ liệu lúc khởi động. Bài §4.4.1.

Hỏi: Vì sao Java thuần và SQLite, không dùng framework?

Nói ngắn: Để bám code mẫu của môn và để mọi bước đều đọc thấy trong code; SQLite là một file nên máy nào cũng chạy được, không cần mạng trừ lúc gọi Gemma. Cái giá: phải tự viết câu SQL, và chưa dành cho nhiều người dùng cùng lúc.

Nếu thầy hỏi tiếp: Máy chủ HTTP có sẵn trong Java; thư viện ngoài chỉ có `sqlite-jdbc`, `gson`, `slf4j` (`demo/lib`). Chi tiết ở QĐ9.

Hỏi: Muốn mở rộng thì làm gì?

Nói ngắn: Nhiều người dùng: thay SQLite bằng cơ sở dữ liệu dạng máy chủ, chỉ viết lại phần cất dữ liệu. Hiểu ảnh tốt hơn: thay phần biến ảnh thành vector. Thanh toán thật: cắm cổng thanh toán sau phần thanh toán đã có. Nhờ các interface, phần điều phối không phải sửa.

Nếu thầy hỏi tiếp: Bản cài đặt mới cho bốn interface repository; thay `VectorIndex` và `ImageService.embed`; lớp con mới của `Payment`.

5.7 Kiểm chứng yêu cầu

Hỏi: Em kiểm chứng các yêu cầu thế nào?

Nói ngắn: Bốn lớp bằng chứng. Một, kiểm thử tự động theo từng nhóm: luật của đơn, giỏ, kho, đặt hàng, quyền xem đơn, tìm kiếm, trợ lý. Hai, một lần chạy thử từ đầu đến cuối trên cơ sở dữ liệu mới, qua đúng các đường web, cả khi có và khi tắt Gemma. Ba, công cụ so code với đặc tả. Bốn, thử ghi dữ liệu sai vào cơ sở dữ liệu và xem nó bị từ chối. Phần AI thì đo riêng bằng bộ câu hỏi.

Nếu thầy hỏi tiếp: 67 kiểm thử (`build/test_results.txt`); 17/17 bước (`tools/e2e_check.py`, `build/e2e_results.txt`); 6/6 câu sai bị từ chối (`build/sql_validation.txt`). Không phải yêu cầu nào cũng có kiểm thử tự động: bảng yêu cầu ở showcase G ghi rõ cái nào chứng minh bằng demo.

Hỏi: Kiểm thử có gọi Gemma thật không?

Nói ngắn: Kiểm thử tự động thì không: em dùng bản giả, lúc trả lời như Gemma, lúc giả như mất kết nối, nên mọi đường lui đều được thử. Lần chạy thử đầu cuối thì gọi Gemma thật. Chất lượng của AI thật đo riêng ở Chương 3.

Nếu thầy hỏi tiếp: Bản giả trong `demo/tests/java/.../Fixtures.java`. Bước đầu cuối "UC10: the assistant looks the order up with a tool" chạy với Gemma thật; hai bước NFR02, NFR12 chạy với bản tắt Gemma.

Hỏi: Kết quả đánh giá AI có đáng tin không?

Nói ngắn: Chưa thể coi là chứng minh: đó là một lần đo trên bộ câu hỏi nhỏ, và Gemma có thể trả lời hơi khác giữa các lần. Em chỉ khẳng định hai điều: tìm bằng ảnh thì Gemma hơn rõ, tìm bằng chữ thì Gemma không hơn luật đơn giản. Số đo làm ngày 30/10 trên máy chủ của lớp; hôm nay demo gọi Gemma 4 qua DeepInfra nên câu chữ có thể khác đôi chút.

Nếu thầy hỏi tiếp: Ảnh 75% so với 12% (n=8); chữ 93% so với 93% (n=14), diễn đạt lại 67% so với 67% (n=6); giọng nói đầu cuối 83% (n=12); trợ lý 7/7 đúng, 0 lần lộ dữ liệu. Bài §3.15.

5.8 Đánh đổi và hạn chế

Hỏi: Hạn chế lớn nhất của bài là gì?

Nói ngắn: Em nói thẳng năm điều, mỗi điều có cách sửa. Đặt hàng chưa nằm gọn trong một giao dịch: nếu sập giữa chừng có thể có đơn đã lưu mà kho chưa trừ; cách sửa là gói lưu đơn, trừ kho, làm rỗng giỏ vào một giao dịch. Hủy đơn chưa trả hàng về kho. Thanh toán chỉ mô phỏng, chưa có mật khẩu. Hiểu ảnh còn đơn giản. Và chỉ chạy trên một máy, một file dữ liệu.

Nếu thầy hỏi tiếp: Bài §5.12; showcase G "Limitations". Cách sửa việc hủy: thêm thao tác ngược với `decreaseStock` rồi gọi trong `OrderService.cancelOrder`.

Hỏi: Vì sao "hiểu ảnh" bằng ba nhãn mà không dùng vector học sâu?

Nói ngắn: Ba nhãn (phong cách, chủ đề, màu) giải thích được cho khách ("cùng phong cách, cùng màu"), dùng chung từ vựng với tìm bằng chữ nên cộng điểm được, và không cần thêm mô hình trên laptop. Cái giá là độ tinh: hai bức phong cảnh xanh khác hẳn nhau vẫn giống nhau với cửa hàng.

Nếu thầy hỏi tiếp: Vector 28 chiều (10 phong cách, 8 chủ đề, 10 màu) do `VectorIndex.toVector` tạo, so bằng cosine. Bảng `product_feature` có cột `model_version` để sau này thay bằng vector học được.

Hỏi: Gemma ngang luật khi tìm bằng chữ, vậy dùng Gemma làm gì?

Nói ngắn: Trên bộ câu hỏi nhỏ của em thì đúng là ngang, và em không khẳng định điều chưa đo được. Giá trị rõ của Gemma là ở ảnh, ở việc đọc căn phòng để chọn tranh, và ở trợ lý biết tự gọi công cụ tra đơn. Còn ở chữ, luật đơn giản đủ tốt và nhanh hơn nhiều, nên nó vừa là đường lui vừa là mốc so sánh.

Nếu thầy hỏi tiếp: Thời gian: 0,44 giây với Gemma, 0,006 giây với luật. Phòng: 12/13 câu giải thích có chi tiết cụ thể.

5.9 Sơ đồ chụp từ bản dùng thử

Hỏi: Vì sao có năm sơ đồ là ảnh chụp bản dùng thử của Visual Paradigm?

Nói ngắn: Bản Community miễn phí từ chối vẽ BPMN, ArchiMate và DFD. Bản dùng thử vẽ được nhưng in dấu "evaluation copy" lên mọi file xuất ra, nên em chụp chính cửa sổ VP và ghi rõ "bản dùng thử" dưới mỗi hình. Mọi sơ đồ UML và ERD còn lại xuất từ bản Community, không có dấu. Cả năm sơ đồ vẫn nằm trong file project, mở và sửa được.

Nếu thầy hỏi tiếp: Năm sơ đồ: 1.3 Stakeholder Map, 1.4 Business Process (BPMN), 1.6 Problem Tree, 1.7 Context Diagram - DFD Level 0, 2.9c DFD Level 1. Hình minh họa trong phụ lục "Từ phân tích đến thiết kế" của thầy cũng là ảnh chụp VP Enterprise (Evaluation Copy). Bản dùng thử 30 ngày từ 5/11 nên hôm nay vẫn mở được; hết hạn thì ảnh chụp và file project vẫn còn, bản Community vẫn mở mọi sơ đồ UML, ERD.

6. Demo trực tiếp

Kể demo như **câu chuyện của một vị khách**: chị ghé Họa., tìm tranh bằng lời và bằng giọng nói, đưa ảnh một bức tranh rồi ảnh phòng khách, đặt mua; người quản lý chuyển đơn đi; chị hỏi trợ lý và tò mò thử xem đơn của người khác; cuối cùng ta rút phích AI. Năm flow, cùng thứ tự với mục F của showcase, tổng khoảng **6 phút**. Trong kịch bản 7 bước (bước 6) chỉ dùng một phần Flow 1, 3 và 4; các flow còn lại dùng khi thầy muốn xem thêm. Mọi câu tìm đều bằng **tiếng Anh**.

Ba điều cần biết trước khi demo

- Gemma có thể trả lời **hơi khác giữa các lần** (câu lý do, thứ tự các bức phía sau). Phần "Mong đợi" chỉ là tinh thần, đừng đọc nguyên văn.
- Giỏ của C001 có sẵn **một bức** (Woman with a Parasol, 89 USD), nên tổng tiền lớn hơn món vừa thêm. Cứ nói "trong giỏ có sẵn một bức".
- Muốn cho thầy thấy AI có tham gia hay không: **Shift+D** bật chế độ nhà phát triển, hiện nhãn "GEMMA 4" hoặc "KEYWORD FALLBACK" và thời gian. Bật khi cần, rồi tắt.

6.1 Flow 1 · Tìm bằng lời và bằng giọng nói (≈ 1 phút; UC02, UC03)

Thao tác

Tab cửa hàng, bấm *Explore the collection*. Gõ `blue landscape paintings`, Enter. Rồi bấm micro trong ô tìm, nói rõ `Monet paintings under 100 dollars`, bấm dừng; chữ hiện trong ô *We heard*, sửa nếu sai, bấm *Search*.

Nói

Một vị khách ghé Họa. Chị không nhớ tên bức tranh, chỉ nhớ đó là phong cảnh, màu xanh. Chị gõ đúng như vậy, và mỗi bức hiện ra kèm một dòng lý do vì sao được chọn. *[bấm micro, nói tiếng Anh]* Lần này chị nói: tranh của Monet, dưới một trăm đô. Giọng nói được đổi thành chữ ngay trên chiếc laptop này, âm thanh không đi đâu cả; chữ hiện ra để chị sửa nếu máy nghe sai. Ba bức đầu đúng là ba bức Monet dưới một trăm đô.

Mong đợi

Chữ: khoảng mười hai bức, bức đầu thường là Woman with a Parasol, mỗi thẻ có dòng lý do. Giọng nói: Impression, Sunrise (79 USD), Water Lilies (69), Woman with a Parasol (89), rồi vài bức Ấn tượng khác dưới 100 USD (cửa hàng chỉ có ba bức Monet đủ điều kiện).

Nếu lỗi

Nhãn "KEYWORD FALLBACK": Gemma không trả lời (mục 6.6). *Voice search is unavailable*: Whisper chưa chạy; gõ câu thay vì nói (mục 7). Máy nghe "Monet" thành "Money": sửa trong ô *We heard*, và nói luôn: đây là lý do khách được xem và sửa chữ.

6.2 Flow 2 · Đưa ảnh, rồi treo tranh lên tường (≈ 1 phút 30; UC04, UC14)

Thao tác

Ở trang Collection bấm *find paintings similar to a photo*, chọn `Q02.jpg` trên Desktop. Rồi kéo thả `room1.jpg` vào trang (hoặc Cmd+V): trang chuyển sang *Find for my room*, bức hợp nhất đã treo sẵn; bấm một bức khác trong

dải *Picked for this wall*.

Nói

Chị có một tấm ảnh chụp bức tranh ở bảo tàng, đây là bức Sóng lừng. Gemma nhìn ảnh và tả nó bằng ba nhãn: phong cách, chủ đề, màu; cửa hàng tìm những bức có nhãn giống nhất, và đúng bức đó đứng đầu. *[kéo ảnh phòng vào]* Rồi chị muốn biết tranh có hợp phòng khách không. Chị đưa ảnh căn phòng: bức hợp nhất được treo ngay lên tường trong ảnh, kèm một câu giải thích nhắc tới những thứ có thật trong phòng. *[bấm bức khác]* Bấm bức khác thì nó treo lên ngay. Ảnh phòng chỉ nằm trong trình duyệt, không được lưu lên máy chủ.

Mong đợi

Ảnh Q02: The Great Wave off Kanagawa đứng đầu. Phòng: "Looking at your room…" vài giây, rồi tranh trên tường và câu *Why it fits*.

Nếu lỗi

Chờ tối đa 15 giây; quá hạn thì cửa hàng tự chọn theo màu và dựng câu giải thích từ thông tin của tranh. Ảnh lỗi định dạng: dùng một trong ba phòng mẫu ở cột bên phải.

6.3 Flow 3 · Mua, xem danh mục, rồi giao hàng (≈ 1 phút 30; UC05, UC07, UC12, UC13)

Thao tác

1. Mở một bức (ví dụ The Great Wave). **Chỉ vào dòng nhỏ trên tên tranh**: *Fine-art prints ▪ Ukiyo-e ▪ 1831*, tức tên danh mục ▪ phong cách ▪ năm. Một cái khung thì dòng này là *Frames ▪ vật liệu*.
2. Chọn một khung, *Add to cart*, mở *Cart*. Chọn địa chỉ, *Cash on delivery*, *Place order*.
3. Bấm tên ở góc phải trên, chọn *Gallery staff*; vào *Back office*. Tab *Paintings & frames*: dưới tên mỗi sản phẩm có tên danh mục ("Fine-art prints ▪ họa sĩ" hoặc "Frames"). Kéo xuống form *Add a painting*: ô *Category* chỉ liệt kê danh mục chứa tranh.
4. Tab *Orders*: ở đơn vừa đặt bấm *Mark processing*. Rồi đổi lại về C001.

Nói

Chị chọn bức này. Dòng chữ nhỏ trên tên tranh là danh mục của nó, chính là dòng trong bảng danh mục em vừa cho thầy xem. *[thêm vào giỏ, đặt hàng]* Chị thêm tranh và khung vào giỏ rồi đặt hàng. Phía sau, cửa hàng đọc lại giá và kho, tạo đơn với giá đã chốt, trừ kho rồi làm rỗng giỏ; tổng tiền đã trừ mười phần trăm cho khách VIP và cộng tám đô phí giao. *[đổi sang Gallery staff]* Bây giờ em là người quản lý. Bảng sản phẩm cũng ghi danh mục của từng món, và khi thêm tranh mới, em chỉ chọn được danh mục dành cho tranh, nên không thể xếp nhầm một bức tranh vào mục khung. *[Mark processing]* Đơn chỉ đi từng bước: mới đặt, đang xử lý, đã gửi, đã giao; mỗi đơn chỉ có đúng một nút cho bước kế tiếp.

Mong đợi

"ORDER PLACED: Your order #… is confirmed". Tổng = tạm tính − 10% + 8 USD. Sau khi bấm, đơn chuyển sang *Processing* và nút đổi thành *Mark shipped*.

Nếu lỗi

Báo hết hàng: dữ liệu còn dư từ lần tập trước; đặt lại dữ liệu mẫu (mục 2.4).

6.4 Flow 4 · Hỏi trợ lý, rồi thử xem trộm (≈ 1 phút; UC10, UC09, UC11)

Thao tác

Là C001: bấm nút chat tròn ở góc phải dưới, hỏi `where is my latest order?`. Rồi mở *My orders*; ở ô *Find an order* gõ `20261003` (đơn của C002), bấm *Look up*. Cuối cùng bấm *Cancel order* ở đơn vừa đặt.

Nói

Chị hỏi trợ lý: đơn mới nhất của tôi đâu rồi? Trợ lý tự gọi công cụ tra đơn và trả lời về đúng đơn vừa đặt, đang được xử lý. Cửa hàng luôn tra theo tài khoản đang đăng nhập, không theo những gì khách gõ. *[tra 20261003]* Giờ chị tò mò gõ mã đơn của một người khác. Cửa hàng trả lời: không tìm thấy trong tài khoản của bạn, giống hệt như khi mã đó không tồn tại, nên không ai dò ra được mã đơn nào là thật. *[Cancel order]* Chị đổi ý và hủy đơn vừa đặt: được, vì đơn chưa gửi đi. Đơn đã gửi thì không còn nút hủy.

Mong đợi

Trợ lý nói về đơn vừa đặt (đang xử lý). Tra 20261003: *"We can't show order #20261003 … was not found in your account."* Hủy: đơn chuyển sang *Cancelled*; đơn 20261001 (đã gửi) không có nút hủy.

6.5 Flow 5 · Rút phích Gemma (≈ 1 phút; NFR02, NFR12)

Thao tác

Sang tab bản tắt Gemma (`http://127.0.0.1:8687/?debug=1`). Gõ lại `blue landscape paintings`; tải `Q02.jpg` ở *find paintings similar to a photo*; hỏi trợ lý `find my order 20261001`.

Nói

Câu hỏi thầy có thể đang nghĩ: nếu AI hỏng thì sao? Đây là một bản của cửa hàng mà đường tới Gemma đã bị cắt. Tìm bằng chữ vẫn ra, chỉ khác nhãn: lần này luật từ khóa làm việc, và còn nhanh hơn. Với ảnh thì khác: cửa hàng chỉ còn đọc được màu trung bình nên trả về bức sai, đúng như em đã đo, Gemma giúp rõ nhất ở ảnh. Trợ lý vẫn tra được đơn nhờ luật. AI hỏng, cửa hàng vẫn bán được hàng.

Mong đợi

Chữ: vẫn ra kết quả, nhãn "KEYWORD FALLBACK" (bản này mở sẵn chế độ nhà phát triển). Ảnh: kết quả xếp theo màu, không phải The Great Wave. Trợ lý: vẫn trả lời đúng đơn; dưới câu trả lời ghi "Fallback ▪ no AI" và "Tool ▪ find_order".

6.6 Khi Gemma không chạy

Nói câu mở ở mục 2.3 trước khi bắt đầu demo, rồi đi các flow như sau. Flow 5 bỏ, vì cửa hàng chính lúc này đã là bản dự phòng.

Flow	Chuyện gì xảy ra	Làm và nói gì
1 · chữ, giọng	Vẫn chạy: luật từ khóa hiểu câu; Whisper vẫn chạy trên laptop	Bật Shift+D, chỉ nhãn "KEYWORD FALLBACK": "đây là đường lui em thiết kế sẵn". Câu Monet vẫn ra đúng ba bức Monet dưới 100 USD.
2 · ảnh, phòng	Ảnh chỉ còn đọc màu, kết quả kém; phòng xếp theo màu, lý do dựng từ thông tin của tranh	Làm nhanh một lần để thấy kết quả kém, rồi mở showcase F ("Show a photo, then hang it on your wall") hoặc `figures/demo/search_image.png`, `room_preview.png`: ảnh chụp khi có Gemma.
3 · mua, giao	Không ảnh hưởng (NFR12)	Làm đầy đủ: đây là phần chứng minh cửa hàng vẫn bán được.
4 · trợ lý	Trợ lý trả lời bằng luật (chế độ nhà phát triển ghi "Fallback ▪ no AI"); quyền xem đơn y nguyên	Hỏi `find my order 20261001` hoặc `where is my latest order?`, vì luật chỉ nhận mã đơn và vài từ khóa; phần tra đơn của người khác vẫn làm như cũ.

Nếu được hỏi "vậy số đo có Gemma ở đâu?": mục G của showcase và bài §3.15 (đo ngày 30/10).

6.7 Nếu thầy muốn xem VP sinh cơ sở dữ liệu ngay tại chỗ

Bài đã có hai ảnh chụp trình hướng dẫn *Generate Database* của chính VP (bài §4.2.3, showcase mục E). Muốn chạy lại trước mặt thầy, chỉ cần một lệnh (khoảng 2 phút):

1. Đóng mọi cửa sổ Visual Paradigm.
2. Ở thư mục dự án chạy `python3 tools/vp_generate_db.py`. Cửa sổ VP (bản dùng thử) tự mở, tự dựng sơ đồ quan hệ thực thể, rồi tự bấm *Tools* → *DB* → *Generate Database*, chọn SQLite, *Generate DDL* và OK. Trong lúc nó chạy, đừng bấm chuột vào cửa sổ VP.
3. Khi xong, lệnh in kết quả so sánh. Mong đợi dòng "Same tables, columns, types, NOT NULL, primary keys, UNIQUE and foreign keys: True (18 tables, 91 columns, 19 foreign keys compared)".
4. Lệnh ghi đè hai ảnh và file `demo/db/vp_wizard_ddl.sql` bằng bản vừa chạy; muốn trả lại bản đã nộp: `git checkout -- figures/vp demo/db`.

Đừng mở `vp/tieuluan_design.vpp` bằng bản dùng thử: nó báo "Evaluation on this project has been expired". Vì

vậy công cụ để VP dựng sơ đồ từ cùng mô hình trong một project mới.

Nói

Em để Visual Paradigm tự sinh cơ sở dữ liệu từ chính sơ đồ quan hệ thực thể, bằng đúng trình hướng dẫn của nó. File nó viết ra giống cơ sở dữ liệu cửa hàng đang chạy ở mọi bảng, cột, khóa chính và khóa ngoại; chỉ thiếu các ràng buộc kiểm tra giá trị, vì sơ đồ không chứa chúng, nên công cụ của em thêm vào từ cùng mô hình. Lần đầu chạy, VP còn bắt được một lỗi của em: sáu khóa ngoại cùng tên “is a”. Em đặt lại tên theo quy ước khóa ngoại bảng cột.

Nếu lỗi

Lệnh báo lỗi hoặc VP không mở: mở hai ảnh trong showcase mục E và file `build/schema_comparison.txt` của lần chạy trước. Có dòng `DIFFERENCE`: đọc to dòng đó và giải thích.

7. Sự cố và cách xử lý

Nguyên tắc: **bình tĩnh và nói thật**, vừa sửa vừa nói cho thầy biết mình đang làm gì. Quá 30 giây thì bỏ qua và mở ảnh chụp: showcase có ảnh của mọi sơ đồ (mục cuối “All diagrams”) và mọi màn hình demo (mục F); ảnh gốc ở `figures/vp/` và `figures/demo/`. Nhiều sự cố của AI lại là dịp chứng minh đường lui (NFR02).

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
Showcase không mở được ở `localhost:8000`	Terminal 3 chưa chạy, hoặc cổng bận	Chạy lại `python3 -m http.server 8001 --bind 127.0.0.1` và mở cổng 8001. Hoặc mở bản trên GitHub Pages (cần Internet).
Không kết nối được `127.0.0.1:8686`	Cửa hàng chưa chạy hoặc đã tắt	Trong `demo` chạy `./run.sh`; kiểm tra bằng `lsof -nP -iTCP:8686 -sTCP:LISTEN`.
Address already in use	Tiến trình cũ còn chạy	`lsof -tiTCP:8686 -sTCP:LISTEN` lấy mã tiến trình, `kill <mã>`, rồi `./run.sh`.
`Gemma 4: offline`, hoặc nhãn “KEYWORD FALLBACK”	Mất Internet; key sai hoặc thiếu; dịch vụ Gemma lỗi	Làm theo mục 2.3: kiểm tra mạng và key, sửa xong thì khởi động lại cửa hàng. Không sửa được thì trình bày dự phòng có chủ đích (mục 6.6).
Gemma chậm	Mạng yếu	Mỗi lần gọi chờ tối đa 15 giây rồi tự chuyển sang đường lui. Trong lúc chờ, kể tiếp điều đang xảy ra phía sau.
Voice search is unavailable	Whisper chưa chạy	`cd demo/voice-server && ./run_voice.sh`, đợi khoảng 10 giây, kiểm tra `curl http://127.0.0.1:9000/health`. Tạm thời gõ câu thay vì nói.
We couldn’t hear you, hoặc nghe sai	Micro tắt; tiếng ồn	Cho phép micro (ổ khóa trên thanh địa chỉ Chrome); nói to, chậm, bằng tiếng Anh; sửa chữ trong ô *We heard*.
Tải ảnh không được	Không phải JPEG, PNG, WebP, hoặc quá 5 MB	Dùng `Q02.jpg`, `room1.jpg` hoặc ba phòng mẫu có sẵn. Nói luôn: đây là NFR03, máy chủ kiểm tra lại cả khi trình duyệt đã kiểm.
Hết hàng, giỏ lạ, “latest order” là đơn cũ	Dữ liệu còn từ lần tập	Đặt lại dữ liệu mẫu (mục 2.4).
VP Community báo sơ đồ *not supported*	Đó là một trong năm sơ đồ BPMN, ArchiMate, DFD	Bình thường. Mở ảnh chụp trong bài hay showcase (câu trả lời ở mục 5.9); bản dùng thử không mở lại được project.
VP Community không mở được project	Bản quyền, hoặc project đang bị một tiến trình dựng sơ đồ giữ (`vp/.vp.lock`)	Đóng mọi cửa sổ VP, xóa `vp/.vp.lock` nếu không còn tiến trình nào chạy, mở lại bằng bản Community. Không chạy `tools/build_vp.py` trong buổi bảo vệ. Nếu vẫn không được: dùng ảnh trong showcase.

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
VP hỏi có lưu không	Đã lỡ kéo một hình	Chọn *Don't Save*: file project phải giữ nguyên như lúc nộp.
`sqlite_web: command not found`	Chưa cài sqlite-web	Dùng `sqlite3 -readonly -header -column demo/data/ecom.db` và các câu ở mục 2.2.
`./run.sh` báo thiếu JDK	Chưa cài Java 17 trở lên	`brew install openjdk@21`, rồi `./build.sh` trong `demo`.
Chữ trên máy chiếu nhỏ	Độ phân giải	Phóng to Chrome (Cmd và dấu +) lên 125%; VS Code: Cmd và dấu +; VP: thanh zoom ở góc dưới.
Mọi thứ hỏng	Máy hoặc mạng	Chỉ cần trình duyệt: showcase (bản trên máy hoặc GitHub Pages) có đủ sơ đồ và ảnh demo. Kể lại năm flow bằng ảnh ở mục F, và nói thẳng rằng demo trực tiếp không chạy được.

Những điều không nên nói

- Không nói Gemma *hơn* luật từ khóa khi tìm bằng chữ: số đo cho thấy ngang. Chỉ nói Gemma hơn rõ ở ảnh.
- Không nói số đo ở Chương 3 là đo trên DeepInfra: chúng đo ngày 30/10 trên máy chủ của lớp.
- Không nói VP sinh ra *toàn bộ* cơ sở dữ liệu: bản của VP thiếu khóa ngoại và CHECK, em bổ sung từ cùng mô hình.
- Không nói mọi yêu cầu đều có kiểm thử tự động; không nói vector ảnh là embedding học được; không nói thanh toán là thật; không nói đặt hàng nằm trong một giao dịch.
- Không mở `demo/config/app.local.properties` khi đang chiếu (có key); không đọc key hay địa chỉ máy chủ lên.

8. Phụ lục

8.1 Những con số cần thuộc

Lấy tự động từ dự án lúc dựng tài liệu, nên khớp với bài, showcase và kết quả chạy.

Chủ đề	Số liệu
Yêu cầu	32: 20 chức năng (12 Must, 6 Should, 2 Could) và 12 chất lượng; 14 use case; 4 tác nhân
Lớp	18 lớp phân tích; 69 lớp thiết kế = 60 lớp Java + 9 view; 14 package trong 3 tầng và miền dùng chung
Cơ sở dữ liệu	18 bảng, 19 khóa ngoại, 20 ràng buộc CHECK; 154 dòng mẫu; 32 tranh, 5 khung, 2 danh mục
So với VP	trình hướng dẫn Generate Database: giống về bảng, 91 cột, kiểu, NOT NULL, khóa chính, UNIQUE, 19 khóa ngoại; chỉ thiếu 20 CHECK
Kiểm chứng	67 kiểm thử đạt (2026-11-12 16:15); chạy thử đầu cuối 17/17 bước; 6/6 câu sai bị từ chối; so khớp: 60 lớp, 169 trường, 185 phương thức, 91 cột, 62 câu SQL
Sơ đồ	40 sơ đồ trong project VP; 5 ảnh chụp bản dùng thử
Code	4621 dòng Java, 2337 dòng JavaScript
AI (đo 30/10)	ảnh 75% và 12% (Hit@1, n=8); chữ 93% và 93% (n=14); Whisper sai 2.0% số từ, trung bình 0,70 giây, tối đa 1,43 giây (n=20); trợ lý 7/7 đúng, 0 lần lộ dữ liệu
Cấu hình	cửa hàng cổng 8686 (bản tắt Gemma 8687), Whisper `127.0.0.1:9000` mô hình *small*; Gemma chờ tối đa 15 giây; trọng số xếp hạng 0,45 / 0,45 / 0,10; phí giao 8 USD; khách VIP giảm 10%

8.2 Hỏi gì thì mở gì

Số sau dấu hai chấm là số dòng (lấy tự động lúc dựng tài liệu). File Java nằm trong `demo/src/main/java/com/ptit/ecom/`, file kiểm thử trong `demo/tests/java/com/ptit/ecom/test/`. "Bài §" là mục trong tiểu luận; "VP" là tên sơ đồ trong project; "SC" là mục của showcase.

Thầy hỏi về	Mở
Vấn đề, mục tiêu, người dùng	SC A; bài §1.2, §1.3, §1.6; VP 1.3 Stakeholder Map, 1.6 Problem Tree (bản dùng thử)
Yêu cầu, MoSCoW, use case	SC A; `tools/model.py`:18 (`REQUIREMENTS`, rồi `USE_CASES`); VP 1.2 Use Case Diagram, 2.4a–c; bài §2.2, §2.3, §2.4
Ranh giới, luồng thông tin	VP 1.1 Context View; 1.7 Context Diagram - DFD Level 0 và 2.9c DFD Level 1 (bản dùng thử); bài §1.7, §2.7
Đặc tả use case, luồng thay thế, luật	SC D (liên kết tới nguồn LaTeX); bài §2.6 (UC07 ở §2.6.4, UC11 ở §2.6.5), luật BR01–BR11 ở §2.8
Sơ đồ hoạt động, tuần tự phân tích	VP 2.2 Activity - Checkout, 2.3 Activity - Multimodal Search, 2.6a–c Analysis Sequence
Mô hình miền, giỏ khác đơn	VP 2.8 Domain Model; bài §2.9; `OrderItem.java`:16
Ba mức dữ liệu	SC C chuỗi 2; VP 2.9a Conceptual, 2.9b Logical, 4.2 Physical Data Model; bài §2.10, §4.2.1
Danh mục và bảng chi tiết	SC E; `ecom_schema_sqlite.sql`:48, 57, 79, 95; `Category.java`:13; `SqliteProductRepository.java`:97, 202; bài §4.2.2
VP sinh cơ sở dữ liệu	ảnh `figures/vp/c42_generate_db.png`, `c42_generate_db_done.png`; `demo/db/vp_wizard_ddl.sql`; `build/schema_comparison.txt`; showcase E; bài §4.2.3; làm trực tiếp: mục 6.7
Khóa, ràng buộc	`ecom_schema_sqlite.sql`:115 (UNIQUE), 125 và 147 (khóa ghép); `build/sql_validation.txt` (câu sai bị từ chối); `Database.java`:25 (bật khóa ngoại)
Lớp thiết kế, phương thức	VP 4.3 Class Diagram in Design; `tools/model.py`:235 (`Order` ở dòng 359); bài §4.3
Ba tầng, package, DAO	VP 4.4a Three Layers, 4.4c Package Diagram, 4.4a Three Logical Components, 4.4b–c Classes per Layer; `tools/model.py`:742; `EcomApplication.java`:40; bài §4.4.1
Đặt hàng	SC B, SC D "Checkout"; VP 4.4d Checkout across the layers; `CheckoutService.java`:45, 100
Hủy đơn, trạng thái đơn	SC D "Cancel Order", "Update Order Status"; VP 2.4 State Machine - Order; `Order.java`:41, 47; `OrderService.java`:40, 51; `DomainTest.java`:78, 85
Quyền xem đơn (NFR01)	`OrderService.java`:21; `Order.java`:55; `WebServer.java`:256 (mã khách từ phiên), 105 (404); `OrderServiceTest.java`:27
Trừ kho	`SqliteProductRepository.java`:70; `ecom_schema_sqlite.sql`:62; `RepositoryTest.java`:82
AI và đường lui	SC G "AI: five features"; VP 3.1 AI Components; `LlmClient.java`:11; `QueryService.java`:73, 93; `ImageService.java`:69, 117; `AssistantService.java`:43, 77, 103; bài §3.13
Giọng nói	`demo/voice-server/server.py`; `WhisperClient.java`:32; `SpeechService.java`:17; bài §3.8
Đánh giá AI	`build/eval/eval_summary.txt`, `build/eval/voice_eval.txt`; bài §3.15
Code khớp mô hình	`tools/check_code_model.py` (`check_database`:232, `check_dependencies`:298); dòng cuối `build/test_results.txt`; bài §4.5.1
Sinh sơ đồ, bảng, code	`tools/build_vp.py`; `tools/gen_sql.py` (`create_table`:69, `validate`:195); `tools/gen_java.py` → `demo/generated/`
Kiểm thử, chạy thử	`build/test_results.txt`; `build/e2e_results.txt`; `tools/e2e_check.py`; bài §5.10
Triển khai, cấu hình	VP 5.1 Deployment; `demo/config/app.properties` (không mở `app.local.properties`); bài §5.2, §5.11
Ảnh bản dùng thử	`figures/vp/README.md`; chú thích các hình 1.3, 1.4, 1.6, 1.7 và DFD mức 1 trong bài
Hạn chế	SC G "Limitations"; bài §5.12
Các đường API	`demo/API.md`